Autonomous Space Debris Detection & Low-Delta V Collision Avoidance

PS17:

1. Executive Summary & Core Concept

Problem: Existing ground-based collision warnings rely on imprecise radar data with massive uncertainty bounds ("error ellipsoids"). Satellites execute premature, high-Δv avoidance maneuvers for false positives, burning critical fuel and shortening operational lifespan.

Solution: An autonomous, zero-added-sensor-mass edge computing pipeline that repurposes onboard Star Trackers and offloads image processing to a dedicated FPGA accelerator.

Operational Workflow

1. **Coarse Warning Ingestion:** Satellite receives ground-station Conjunction Data Messages (CDMs) flagging potential high-risk conjunctions weeks in advance.
2. **Predictive Alignment:** Satellite uses reaction wheels to slew its orientation, pointing an existing Star Tracker along the predicted threat approach vector during non-mission orbits.
3. **Parallel Edge Processing:** The raw Star Tracker video stream is tapped into an independent FPGA module to prevent hijacking the Attitude Determination and Control System (ADCS).
4. **Transient Streak Extraction:** The FPGA runs real-time frame differencing to strip static background stars and isolate high-velocity debris streaks.
5. **3D State Estimation:** An onboard Extended Kalman Filter (EKF) converts 2D angular streak data across multiple frames into a precise 3D trajectory.

6. **Intersection Verification:** System checks if the 3D trajectory intersects the satellite's defined safety volume (the "pizza box").
7. **Proactive Burn Execution:** If collision probability exceeds threshold, the satellite calculates and executes a minimal Δv maneuver at the optimal orbital node long before closest approach.

2. Active-Active Avoidance Protocol (Operational Spacecraft vs. Spacecraft)

When a predicted conjunction involves two fully functional, maneuverable satellites, executing double burns or incorrect maneuvers escalates collision risk. The onboard system runs a multi-variable decision engine to determine which spacecraft executes the maneuver.

Decision Matrix & Scoring Function

Both spacecraft exchange minimal state packets via ground uplink or inter-satellite links. The system calculates a **Maneuver Responsibility Score (S)** based on four weighted metrics:

$$S = w_1 \cdot \text{Cost}_{lifetime} + w_2 \cdot \text{Cost}_{\Delta v} + w_3 \cdot \text{Cost}_{priority} + w_4 \cdot \text{Cost}_{downtime}$$

The satellite with the **lower score** performs the maneuver.

1. **Remaining Delta-V Reserves (Δv Margin):** The satellite with greater remaining fuel reserves assumes priority to execute the burn.
2. **Remaining Operational Lifetime:** A satellite near End-of-Life (EOL) preserves its remaining fuel; the younger asset with longer operational margin takes the burn.
3. **Mission Priority / Payload Criticality:** High-value national defense or primary scientific payloads are prioritized to remain on track over secondary or commercial payloads.
4. **Operational Downtime / Recovery Latency:** Measures the time required for a satellite to re-orient, re-stabilize, and resume revenue/mission operations post-burn. The spacecraft with the shorter operational recovery time executes the burn.

The Ground-Station AI (The Decoy)

The Objective: Predict atmospheric drag fluctuations to tighten the error ellipsoid before uplinking the data to the satellite.

The Stack: extremely lightweight. A massive Deep Neural Network (DNN) is not needed. It will take too long to train and will overfit. Use a standard **XGBoost** regressor or a simple **LSTM** (Long Short-Term Memory) time-series model in Python.

The Inputs (The Features):

1. **Space Weather Data:** Pull the F10.7 solar flux and Kp index (geomagnetic activity) APIs from NOAA. Solar flares expand the atmosphere and cause sudden drag.
2. **Historical SSA Data:** Pull historical Two-Line Elements (TLEs) from Space-Track.org to track how specific debris orbits have degraded in the past.

The Output: The model spits out a predictive drag coefficient. Your Python dashboard uses this to calculate a refined Conjunction Data Message (CDM) with a much smaller error margin.

The Integration: The dashboard displays the 3D orbit and the "AI Risk Score" . It then uplinks that refined CDM to your satellite. The Star Tracker and FPGA take that refined data, look at the actual physical space, and make the final deterministic burn decision.

3. Critical Edge Cases & Technical Mitigations

- **Eclipse / Solar Shadow (Optical Blindness):** Debris reflects sunlight; in Earth's shadow, optical detection fails.
    - *Mitigation:* The orbit propagator evaluates solar illumination vectors. If the conjunction occurs in darkness, the system automatically falls back to ground-based CDM data to execute a conservative burn.
- **High Relative Velocity & Streak Smearing:** Head-on approaches ($>15\text{ km/s}$) turn point sources into faint, spread-out streaks that standard star

algorithms discard as noise.

    - *Mitigation:* FPGA runs a line-detection pipeline (e.g., parallelized Hough transform) tuned specifically for multi-frame transient line detection.
- **Sensor Saturation (Sun/Earth Limb Blinding):** Pointing star trackers near the Sun or Earth's atmospheric limb washes out the sensor.
    - *Mitigation:* The slew planner checks solar/nadir exclusion zones before reorienting the spacecraft. If unviable, the maneuver defaults to ground estimates.

- **ADCS Computation Hijack:** Diverting primary MCU cycles to run heavy vision algorithms causes attitude lock drop.
    - *Mitigation:* Hardware isolation. Raw pixel feeds stream directly to an FPGA co-processor via dual-port RAM/DMA, leaving the ADCS flight MCU dedicated to attitude control.
- **2D-to-3D Depth Ambiguity:** A single optical sensor provides angular position, not direct range.
    - *Mitigation:* Implements Angles-Only Orbit Determination (AOOD) fused with multi-frame EKF to resolve orbital elements and range over consecutive orbit passages.
- **Mission Slew Trade-off Cost:** Rotating the satellite to look for debris disrupts Earth-pointing primary payloads.
    - *Mitigation:* An algorithmic trade-off engine calculates if the revenue/data loss of a slew check is lower than the expected fuel cost of an unconfirmed blind burn.

4. Project Development Roadmap & Task Breakdown

Phase 1: Physics, Environment & Orbital Simulation Engine

- **Orbital Mechanics Core:** Implement a 2-body orbital propagator with J_2 Earth oblateness perturbations in Python.

- **Celestial Background Generator:** Build a static celestial sphere rendering cataloged star maps, constellations, Sun, and Moon vectors.
- **Lighting & Shadow Pipeline:** Calculate solar illumination vectors, Earth shadow (umbra/penumbra) boundaries, and glare limits.
- **Spacecraft & Sensor Kinematics:** Model reaction wheel torque, spacecraft slew rates, and Star Tracker Field of View (FOV) geometry ($15^\circ \times 15^\circ$ cone).

Phase 2: Star Tracker Optical Simulation & Sensor Modeling

- **Frame Generator:** Synthesize optical frame outputs replicating sensor resolution, dark current noise, exposure times, and integration blur.
- **Streak Dynamics Engine:** Render moving debris streaks based on relative velocity vectors, apparent magnitude, and integration time.

Phase 3: FPGA Hardware Co-Processor (Verilog/VHDL)

- **I/O Pipeline:** Design UART/SPI/Ethernet wrappers to stream simulated video frames into the physical FPGA dev board (Artix-7).
- **Frame Differencing Core:** Implement parallel line-buffer memory structures for high-speed background subtraction.
- **Feature Extraction Unit:** Build Verilog logic to extract centroid coordinates and streak vectors from raw pixel streams in real-time.
- **Control Bus Interface:** Output filtered streak data back to the primary system via AXI/SPI bus.

Phase 4: Core Algorithmic Suite

- **Angles-Only Orbit Determination (AOOD):** Implement Gauss/Laplace algorithms to convert line-of-sight angular vectors into preliminary orbit state vectors.
- **Extended Kalman Filter (EKF):** Write recursive state-estimation routines to refine debris range, velocity, and orbital parameters over time.
- **Collision Geometry Engine:** Compute distance of closest approach (DCA) and collision probability against the spacecraft's safety bounding box.

- **Active-Active Decision Engine:** Code the multi-variable scoring model for operational satellite-to-satellite burn negotiation.
- **Trajectory Optimization Routine:** Calculate minimum Δv impulsive burn vectors at optimal true anomaly locations.

Phase 5: Dataset & Test Automation

- **Synthetic Data Pipeline:** Generate structured testing datasets containing varying streak velocities, lighting conditions, background noise levels, and mock CDM inputs.
- **Hardware-in-the-Loop (HITL) Bench:** Link the Python simulation environment directly to the physical FPGA development board to run real-time hardware validation demos.